

after install hadoop, spark, kafka
i learn hadoop, spark and make small projects using
[hadoop, spark] and save the output as parquet file
what should learn in apache kafka to make projects using
hadoop, spark, kafka!

Yes. Given your current path, you **do not need to learn all of Kafka before starting projects**.

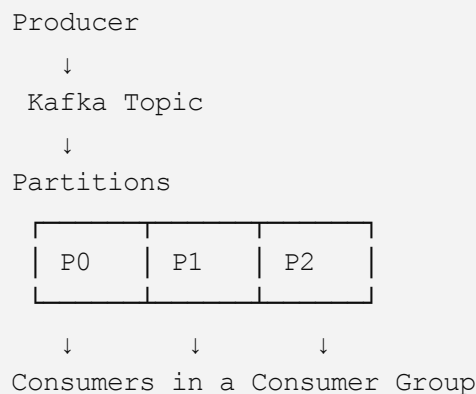
Since you already know Hadoop + Spark and have produced Parquet outputs, I would learn Kafka in this order:

1. Kafka fundamentals — MUST learn

Understand these very well:

- Kafka architecture
- **Broker**
- **Cluster**
- **Topic**
- **Partition**
- **Producer**
- **Consumer**
- **Consumer Group**
- **Message/Event**
- **Offset**
- **Replication**
- **Leader / Follower**
- Why Kafka uses **partitions**
- How partitions provide **parallelism and scalability**
- How consumer groups provide **parallel consumption**

The most important relationship to understand is:



2. Kafka CLI — MUST learn

Before writing Python Kafka code, I strongly recommend becoming comfortable with Kafka from the Ubuntu terminal.

Learn:

```
kafka-topics
kafka-console-producer
kafka-console-consumer
kafka-consumer-groups
```

Practice:

- Create topic
- List topics
- Describe topic
- Delete topic
- Produce messages
- Consume messages
- Consume from beginning
- Create consumer groups
- Check consumer-group offsets
- Understand **current offset**
- Understand **log end offset**
- Understand **consumer lag**

For example:

Bash

```
kafka-topics.sh --create ...
kafka-topics.sh --list
kafka-topics.sh --describe ...

kafka-console-producer.sh ...
kafka-console-consumer.sh ...

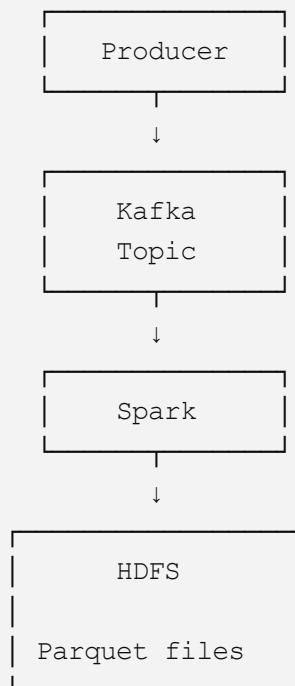
kafka-consumer-groups.sh --describe ...
```

You should be able to operate a small Kafka environment from the terminal before moving on.

3. Kafka + storage — very important for your projects

This is where Kafka starts fitting your **Hadoop/Spark** knowledge.

Understand this architecture:

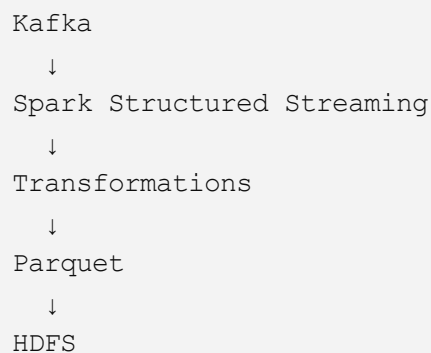


This is very close to the kind of Big Data project you want to build.

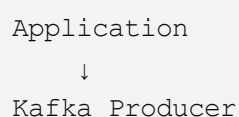
4. Kafka + Spark — MUST learn for your goal

This is the **most important Kafka integration for you.**

Learn how Spark reads from Kafka:



For example:



```
↓  
Kafka Topic: transactions  
↓  
Spark Structured Streaming  
↓  
filter()  
select()  
withColumn()  
groupBy()  
↓  
Parquet  
↓  
HDFS
```

You **do need to learn Spark Structured Streaming** if you want to build proper Kafka + Spark streaming projects.

But you don't need to master every advanced Structured Streaming feature before starting.

Start with:

- Streaming DataFrame
- `readStream`
- Kafka source
- Kafka bootstrap servers
- Topic
- Kafka `key`
- Kafka `value`
- Kafka `partition`
- Kafka `offset`
- `writeStream`
- Checkpointing
- Output modes
- Trigger
- Writing to Parquet
- Reading/writing Kafka data
- Basic streaming transformations

5. Kafka message format — learn this

Don't stay with simple strings forever.

Learn how real events look:

JSON

```
{
  "user_id": 101,
  "product_id": 55,
  "amount": 250.50,
  "timestamp": "2026-08-16T10:30:00"
}
```

Then understand:

```
JSON
  ↓
Kafka
  ↓
Spark
  ↓
parse JSON
  ↓
DataFrame
  ↓
transform
  ↓
Parquet
```

Later you can learn **Avro** and **Schema Registry**, but they aren't required for your first projects.

6. Kafka concepts to learn after your first project

Once you can build a basic Kafka → Spark → HDFS/Parquet project, learn:

- Kafka retention
- Log segments
- Consumer offsets
- Consumer lag
- Delivery semantics
- At-most-once
- At-least-once
- Exactly-once concepts
- Acknowledgments (`acks`)
- Producer retries
- Idempotent producer
- Partition keys
- Ordering

- Rebalancing
- Consumer-group rebalancing
- Kafka Connect
- Schema Registry

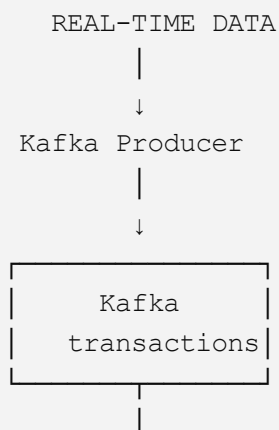
These are more useful once you've actually built something.

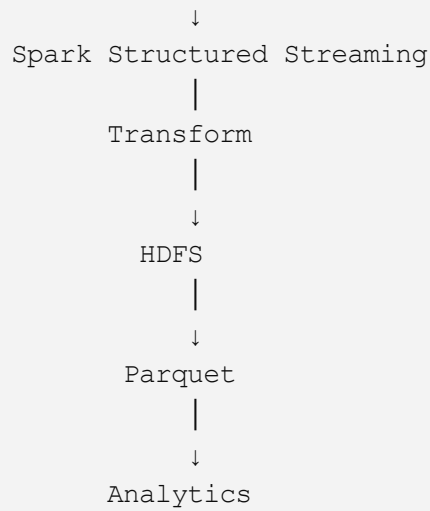
Your learning path

For **your specific goal**, I'd use:

```
Hadoop
  ↓
HDFS
  ↓
Spark / PySpark
  ↓
Batch projects
  ↓
Parquet
  ↓
Kafka fundamentals
  ↓
Kafka CLI
  ↓
Kafka producers/consumers
  ↓
Spark Structured Streaming
  ↓
Kafka + Spark
  ↓
HDFS + Parquet
  ↓
Airflow
```

And your first serious project could look like:





What I would NOT learn yet

Don't get overwhelmed with:

- Kafka internals at source-code level
- Advanced Kafka Streams
- Complex Kafka Connect configurations
- Kubernetes + Kafka
- Multi-datacenter Kafka
- Advanced security
- Schema Registry internals

First get to the point where you can independently build:

Kafka → Spark Structured Streaming → transformations → HDFS → Parquet

That combination is an excellent next step after the Spark batch projects you've already been doing.

Today 2:31 PM

in one topic write me all should i have learn in kafka before projects

Apache Kafka — Everything You Should Learn Before Starting Projects

Since your goal is to build projects using **Kafka + Spark + Hadoop/HDFS + Parquet**, you should learn the following Kafka topics before starting projects.

1. Kafka Fundamentals

- What is Apache Kafka?

- Why Kafka is used
- Kafka architecture
- Kafka cluster
- Broker
- Topic
- Partition
- Producer
- Consumer
- Consumer Group
- Message/Event
- Offset
- Replication
- Leader
- Follower
- ISR (In-Sync Replicas)
- Partitioning
- Why Kafka uses partitions
- How partitions provide parallelism
- How partitions provide scalability
- How replication provides fault tolerance
- Ordering of messages

2. Kafka Installation & Basic Operation

- Kafka installation on Ubuntu
- Starting/stopping Kafka
- Kafka server configuration
- Bootstrap servers
- Understanding Kafka logs/data directories
- Basic Kafka configuration

3. Kafka CLI

Learn to use the Kafka CLI from Ubuntu terminal:

- `kafka-topics`
- `kafka-console-producer`
- `kafka-console-consumer`
- `kafka-consumer-groups`

Practice:

- Create topics
- List topics

- Describe topics
- Delete topics
- Create topics with multiple partitions
- Produce messages
- Consume messages
- Consume from the beginning
- Create consumer groups
- Describe consumer groups
- Check offsets
- Understand consumer lag

4. Producers

Understand:

- How producers send messages
- Producer → topic → partition
- Message keys
- How Kafka chooses a partition
- Partition key
- Ordering
- Producer acknowledgments (`acks`)
- Producer retries
- Idempotent producer
- Basic producer configuration
- Serialization

5. Consumers

Understand:

- How consumers read messages
- Consumer groups
- Multiple consumers in a group
- Consumer-to-partition relationship
- Consumer offsets
- Offset commits
- Automatic vs manual commits
- Consumer lag
- Consumer rebalancing
- Partition assignment

6. Kafka Message Structure

Understand:

Message/Event

- |— Key
- |— Value
- |— Timestamp
- |— Headers
- |— Offset

Learn:

- Key
- Value
- Timestamp
- Headers
- Offset
- Serialization
- Deserialization

Practice with:

- String messages
- JSON messages

For example:

JSON

```
{  
  "user_id": 101,  
  "product_id": 50,  
  "amount": 250.50  
}
```

7. Kafka Storage & Reliability

Understand:

- Kafka logs
- Log segments
- Retention
- Retention by time
- Retention by size
- Replication
- Leader/follower
- ISR
- Fault tolerance

- Durability
- Acknowledgments

8. Kafka + Python

You don't need to become an advanced Kafka-Python developer.

Learn enough to create:

Producer

```
Python Application
    ↓
Kafka Producer
    ↓
Kafka Topic
```

and:

Consumer

```
Kafka Topic
    ↓
Kafka Consumer
    ↓
Python Application
```

Learn:

- Kafka Python client
- Producer
- Consumer
- Sending messages
- Receiving messages
- JSON serialization/deserialization
- Consumer groups
- Basic configuration

9. Kafka + Spark ★ VERY IMPORTANT FOR YOU

Because you want to combine Kafka with Spark, learn:

```
Kafka
    ↓
Spark Structured Streaming
    ↓
Transformation
    ↓
```

HDFS

↓

Parquet

Learn:

- Spark Structured Streaming basics
- Kafka as a streaming source
- `readStream`
- Kafka bootstrap servers
- Kafka topic
- Kafka key/value
- Kafka partition
- Kafka offset
- `writeStream`
- Checkpointing
- Output modes
- Trigger
- Streaming transformations
- Writing streaming results to Parquet
- Reading Kafka data with Spark
- Writing processed data to HDFS

10. Important Kafka Concepts Before Projects

Make sure you understand these clearly:

Topic

↓

Partitions

↓

Offsets

↓

Consumer Groups

↓

Parallelism

↓

Consumer Lag

↓

Replication

↓

Fault Tolerance

Especially understand:

One partition can be consumed by only one consumer within the same consumer group at a time.

This is fundamental to understanding Kafka parallelism.

11. Learn Later — Not Required Before Your First Projects

You can postpone:

- Kafka Connect
 - Kafka Streams
 - Schema Registry
 - Avro
 - Protobuf
 - Advanced security
 - SASL
 - SSL/TLS configuration
 - Kubernetes + Kafka
 - Advanced cluster administration
 - Multi-datacenter Kafka
 - Advanced performance tuning
-

Your final Kafka checklist

Before starting projects, you should be comfortable with:

- ☒ What is Kafka?
- ☒ Kafka architecture
- ☒ Broker
- ☒ Cluster
- ☒ Topic
- ☒ Partition
- ☒ Producer
- ☒ Consumer
- ☒ Consumer Group
- ☒ Message/Event
- ☒ Offset
- ☒ Replication
- ☒ Leader
- ☒ Follower
- ☒ ISR
- ☒ Partitioning
- ☒ Parallelism
- ☒ Scalability
- ☒ Fault tolerance
- ☒ Kafka CLI
- ☒ Create/List/Describe/Delete topics
- ☒ Produce messages
- ☒ Consume messages
- ☒ Consumer groups

- ☑ Consumer offsets
- ☑ Consumer lag
- ☑ Producer basics
- ☑ Consumer basics
- ☑ Message keys
- ☑ JSON messages
- ☑ Serialization/Deserialization
- ☑ Retention
- ☑ Basic Kafka configuration
- ☑ Python Producer
- ☑ Python Consumer
- ☑ Spark Structured Streaming
- ☑ Spark + Kafka
- ☑ Kafka → Spark → HDFS
- ☑ Kafka → Spark → Parquet

Then start projects

Your first project can be:

```
Python Producer
  ↓
  Kafka
  ↓
Spark Structured Streaming
  ↓
Transform / Clean / Aggregate
  ↓
  HDFS
  ↓
  Parquet
```

That's the **core Kafka knowledge you need for your Hadoop + Spark + Kafka projects**.
You don't need to finish every advanced Kafka topic before building.

ChatGPT can make mistakes. Check important info.